

Behavior-Only Window Size Profiling

不依赖任何模型参数，只根据被试的行为正确率序列，为每个被试估计一个适合用于后续 accuracy/error 计算的滑窗大小。

核心思想

对每个候选 $\text{window_size} = w$ ，算法计算一个 behavior-only score：

$$\text{score}(w) = \text{sampling_var}(w) + \text{learning_var}(w)$$

其中：

- $\text{sampling_var}(w)$ 表示滑窗平均带来的二项采样不确定性。
- $\text{learning_var}(w)$ 表示这个窗口内部包含了多少真实学习变化。

直觉上，小 window 的时间分辨率高，但每个 accuracy 是由很少 trial 平均得到的，采样噪声大；大 window 的采样噪声小，但可能把学习过程中过快变化的行为模式抹平。这个 score 试图在二者之间找一个折中。

当前 notebook 中的默认推荐结果使用 $\lambda = 1$ ：

$$\text{score_lambda1}(w) = \text{sampling_var}(w) + \text{learning_var}(w)$$

后来为了检查 learning term 的影响，又额外生成了 $\lambda = 2$ 和 $\lambda = 3$ 的图：

$$\text{weighted_score}(w) = \text{sampling_var}(w) + \lambda * \text{learning_var}(w)$$

候选 Window Size

当前候选窗口集合是：

```
CANDIDATE_WINDOWS = [  
    4, 6, 8, 10, 12, 14, 16,  
    20, 24, 28, 32,  
    40, 48, 64, 80, 96,  
    128, 160, 192, 256, 320, 384, 512  
]
```

但不是每个被试都会评估所有候选值。每个被试有自己的最大允许窗口：

```
max_window = int(n_trials * MAX_FRACTION)
```

其中：

```
MAX_FRACTION = 1 / 3
```

也就是说，候选 window 不能超过该被试总 trial 数的三分之一。例如：

- $n_trials = 320$ 时，最大允许 window 约为 106，所以最大实际候选通常是 96。
- $n_trials = 768$ 时，最大允许 window 是 256，所以可以评估到 256。

此外，还要求每个候选 window 至少能产生一定数量的 rolling accuracy 点：

```
MIN_WINDOWS = 8
```

如果某个 w 下 rolling accuracy 点数少于 8，则这个候选窗口会被跳过。

Rolling Accuracy

对一个被试的正确/错误序列：

$y_1, y_2, \dots, y_n$

其中 y_t 为 0 或 1。对候选窗口 w ，算法计算 overlapping rolling mean：

$$\hat{p}_t(w) = \text{mean}(y_t, y_{t+1}, \dots, y_{t+w-1})$$

一共会得到：

$$n - w + 1$$

个 rolling accuracy 点。

在代码里由 `rolling_mean` 实现：

```
p_hat = rolling_mean(y, window)
```

这里使用的是重叠滑窗，不是 non-overlap bin。因此相邻 accuracy 点并不独立。

Sampling Variance

对每个 rolling accuracy 点 $p_hat_t(w)$ ，算法近似认为它是 w 次 Bernoulli 试验的平均值，因此其采样方差近似为：

$$p_hat_t(w) * (1 - p_hat_t(w)) / w$$

然后对所有 rolling accuracy 点取平均：

$$sampling_var(w) = mean_t[p_hat_t(w) * (1 - p_hat_t(w)) / w]$$

代码中是：

```
sampling_var = float(np.nanmean(p_hat * (1.0 - p_hat) / window))
```

这个项通常会随着 w 变大而下降。原因很直接：window 越大，每个 accuracy 平均了越多 trial，二项采样噪声越小。

Pilot Learning Curve

为了估计一个 window 内部有多少学习变化，算法先构造一条较平滑的行为学习曲线，叫 `pilot_curve`。

它不是最终要使用的 accuracy 曲线，而只是为了帮助评估不同候选 window 的 learning variation。

pilot window 的大小由被试 trial 数决定：

```
pilot_window = int(max(8, min(64, round(n_trials * 0.08))))
```

也就是：

- 至少为 8
- 最多为 64
- 默认约为该被试 trial 数的 8%

然后用 centered rolling mean 得到 pilot curve：

```
pilot_curve = (  
    pd.Series(y)  
    .rolling(  
        window=pilot_window,  
        center=True,  
        min_periods=max(2, pilot_window // 4)  
    )  
    .mean()  
    .interpolate(limit_direction='both')  
    .to_numpy(dtype=float)  
)
```

边缘位置因为 centered window 不完整，所以使用 min_periods 和 interpolation 处理。

Learning Variance

对于候选窗口 w ，算法在 `pilot_curve` 上计算每一个长度为 w 的窗口内部方差：

```
var(pilot_curve_t, ..., pilot_curve_{t+w-1})
```

然后对所有窗口取平均：

```
learning_var(w) = mean_t[var(pilot_curve_t, ..., pilot_curve_{t+w-1})]
```

代码中是：

```
learning_var = float(np.nanmean(rolling_var(pilot_curve, window)))
```

这个项的直觉是：如果一个 window 内部包含明显的学习变化，那么这个 window 太大，会把学习过程的不同阶段混在一起。因此 $\text{learning_var}(w)$ 会变大。

如果某个被试行为曲线很平，或者学习变化主要发生得很慢，那么 $\text{learning_var}(w)$ 可能不会随 w 明显上升。这时总 score 容易偏向较大的 window，因为 $\text{sampling_var}(w)$ 会持续下降。

选择规则

对每个被试，先找到最小 score：

```
min_score = profile['score'].min()
```

然后找出所有在最小值 5% 以内的候选 window：

```
WITHIN_PCT = 0.05  
eligible = profile[profile['score'] <= min_score * (1.0 + WITHIN_PCT)]
```

最后选择这些候选 window 中最小的一个：

```
selected_window = eligible.sort_values('candidate_window').iloc[0]['candidate_window']
```

因此，算法不是简单选择 score 绝对最低的窗口，而是选择“接近最优但尽量小”的窗口。这样做是为了避免在 score 差异很小的时候过度偏向大窗口。

Lambda 的作用

默认 $\lambda = 1$ 时：

$$\text{score}(w) = \text{sampling_var}(w) + \text{learning_var}(w)$$

如果觉得推荐 window 过大，可以提高 λ ：

$$\text{score}(w) = \text{sampling_var}(w) + \lambda * \text{learning_var}(w)$$

提高 λ 的含义是：让算法更重视窗口内部的学习变化，从而更强地惩罚过大的 window。

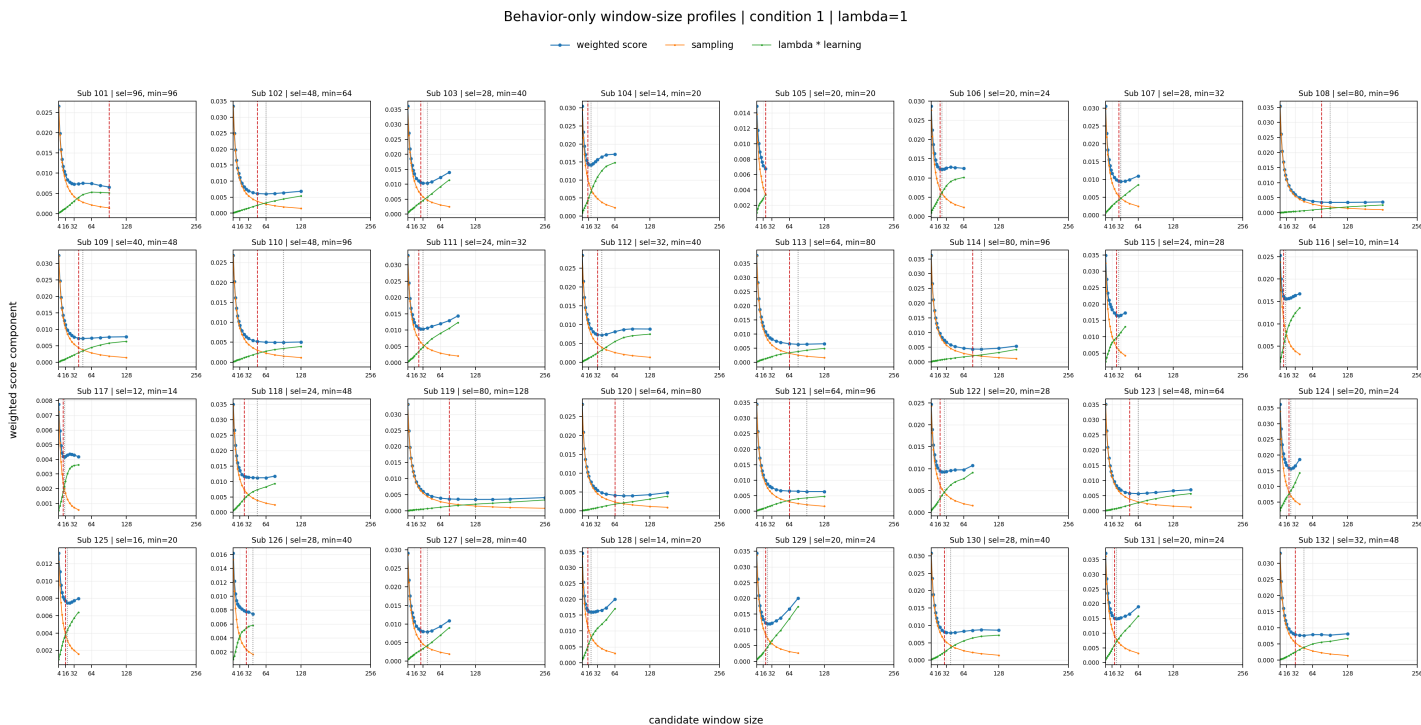
从当前数据的快速敏感性分析看：

lambda=1: cond1 median=28, cond2 median=64, cond3 median=48
lambda=2: cond1 median=20, cond2 median=40, cond3 median=36
lambda=3: cond1 median=15, cond2 median=40, cond3 median=28

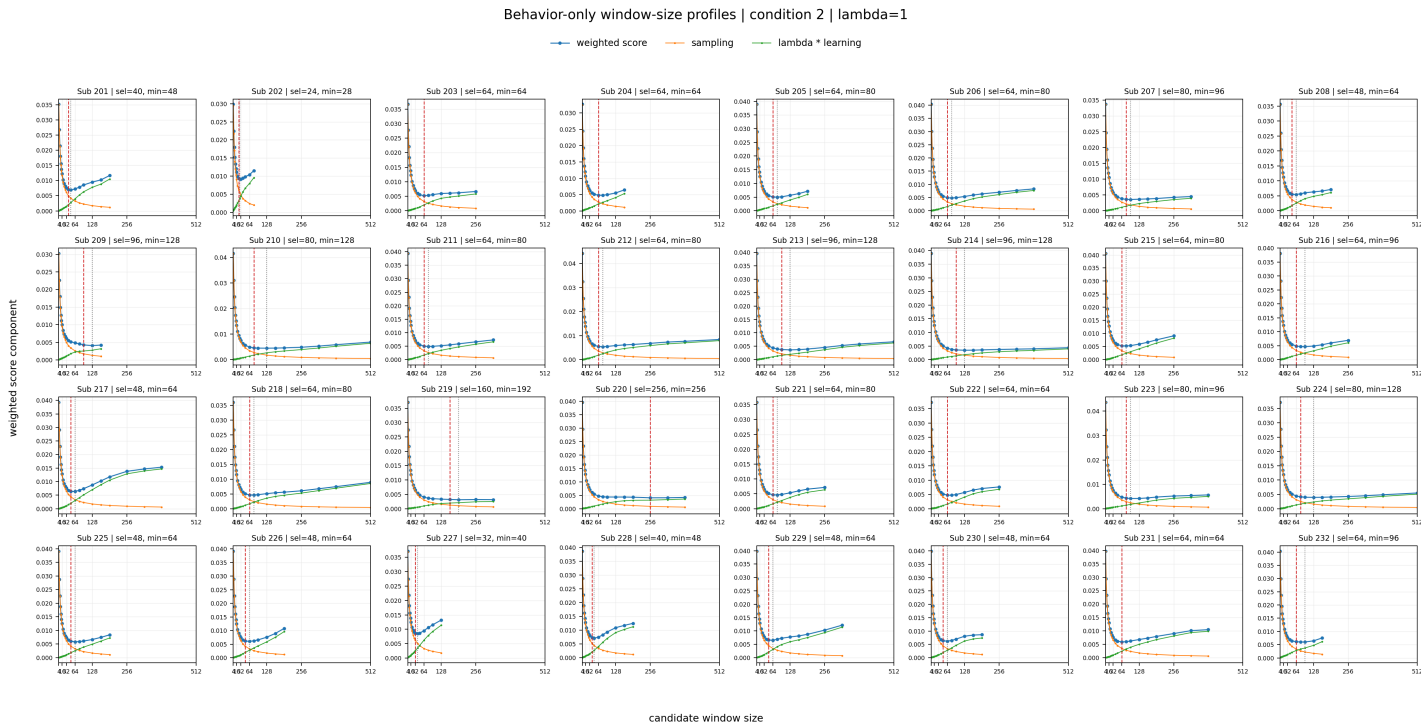
因此， lambda = 2 或 lambda = 3 会明显降低推荐 window，尤其可以减少极端大 window 的情况。

不过， lambda 越大并不一定越好。过大的 lambda 可能会让 window 过小，使 accuracy 曲线过于 noisy。更合理的做法是把 lambda = 1/2/3 作为候选策略，比较它们后续用于模型拟合时的表现。

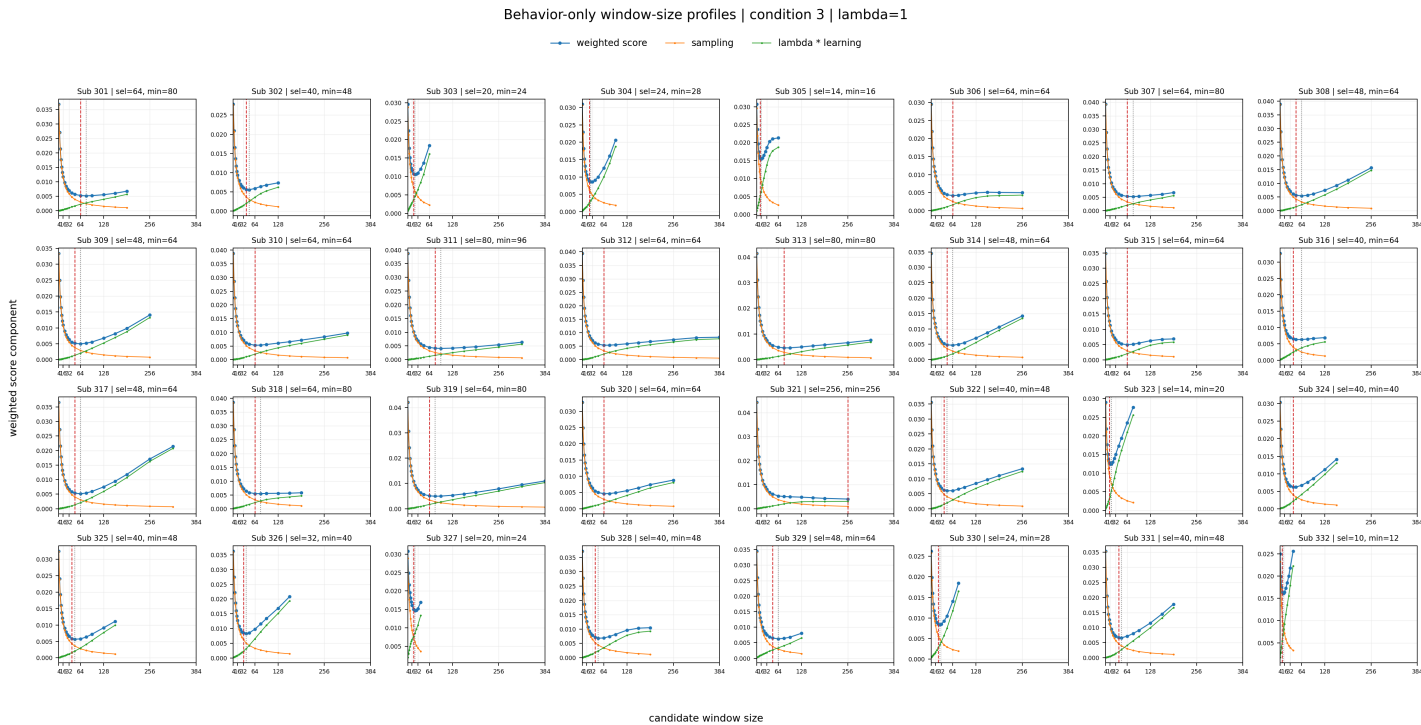
cond1结果



cond2结果



cond3结果



当前方法的优点

这个方法有几个优点：

1. 不依赖模型超参，因此不会被当前模型结构或 hyper-opt 结果直接影响。
2. 每个被试单独估计 window，允许个体差异。
3. 同时考虑小窗口的采样噪声和大窗口对学习变化的平滑。
4. 完整保存了每个候选 window 的 score component，方便之后调整 λ 或选择规则。

当前方法的局限

当前方法也有明显局限：

1. `sampling_var` 和 `learning_var` 的量纲和尺度未经过严格校准， λ 需要经验选择或后续验证。
2. 默认 score 没有显式惩罚“有效 accuracy 点太少”的问题。虽然 `MAX_FRACTION = 1/3` 和 `MIN_WINDOWS = 8` 做了基本约束，但对于 trial 数很多的被试，仍可能选到很大的 window。
3. rolling windows 是高度重叠的，因此 $n - w + 1$ 个 accuracy 点不能当作完全独立的信息点。
4. behavior-only profiling 只看行为曲线本身，没有直接验证这个 window 是否最适合模型拟合。
5. 对学习很慢或行为曲线很平的被试，`learning_var` 可能无法有效阻止 window 继续变大。

推荐的理解方式

当前方法给出的 window size 应该理解为一个 behavior-only 的初步建议，而不是最终真理。

如果目标是后续模型拟合，尤其是用 accuracy error 作为 loss，那么更应该关注：

- 这个 window 下还能保留多少有效行为信息。
- 生成的 accuracy curve 是否能反映被试真实学习 pattern。
- 基于这个 window 优化出来的模型超参是否稳定。
- 不同 λ 下的结论是否一致。

因此，实际使用时可以考虑：

1. 先用 $\lambda = 1/2/3$ 做 sensitivity check。
2. 对极端大 window 的被试单独检查曲线。
3. 如果大 window 导致有效 accuracy 点过少，可以加入额外约束，例如要求：

`n_trials / window_size >= 16`

或：

```
n_trials / window_size >= 20
```

这相当于要求每个被试至少保留足够多的有效行为片段。